

COMPTE RENDU — ÉTAPE 2

Organisation des données

PostgreSQL + ElasticSearch + Kibana via Docker

Étudiant	Hamzaoui Zineddine
Étudiant	Palissier Thomas
Cursus	Data Engineer
Deadline	19 Juin 2025
Bases de données	PostgreSQL + ElasticSearch
Infrastructure	Docker Desktop (3 conteneurs)
Résultat	10 664 entreprises en SQL + 592 avis en ES

1. Contexte et objectif

L'étape 1 consiste à mettre en place une collecte automatisée de données réelles issues des avis clients publiés sur Trustpilot. L'objectif est de récupérer des indicateurs liés à la satisfaction client dans le contexte de la supply chain, afin de constituer une base exploitable pour les phases d'analyse et de visualisation.

Deux types de données collectées

Type 1 — Données agrégées : entreprises par catégorie (note, nombre d'avis, % par étoile) → `companies_trustpilot.csv`

Type 2 — Données détaillées : commentaires clients de ShowroomPrivé (>10 000 avis) → `showroom_reviews.json`

2. Outils utilisés

Installation des librairies Python nécessaires :

```
pip install selenium webdriver-manager beautifulsoup4
```

Outil	Rôle
Selenium	Pilote Chrome automatiquement (contourne la protection 403 de Trustpilot)
webdriver-manager	Installe automatiquement le pilote Chrome compatible
BeautifulSoup	Analyse le HTML et extrait les données JSON de la page

Pourquoi utiliser Selenium plutôt que Requests ?

Lors des premiers tests avec la bibliothèque Requests, l'accès aux pages Trustpilot était bloqué par une réponse HTTP **403 (Forbidden)**, indiquant que les requêtes automatisées étaient filtrées.

Nous avons donc choisi d'utiliser **Selenium**, qui permet de piloter un navigateur réel (Chrome) et de reproduire un comportement proche d'une navigation utilisateur classique. Cette approche permet de charger dynamiquement les pages via JavaScript et d'accéder aux données nécessaires lorsque les requêtes HTTP classiques ne suffisent pas.

3. Source 1 — Entreprises par catégorie

3.1 Données collectées

Catégorie	Entreprises collectées	Trustscore moyen
atm (banques)	47	2.14
clothing_store (boutiques)	9 969	3.57
courier_service (livraison)	648	2.27
TOTAL	10 664	—

3.2 Champs collectés par entreprise

Champ	Exemple
company_name	DHL
trustscore	3.2
nb_avis_total	42 103
pct_excellent	52.1 %
pct_bad	20.5 %
date_scraping	2026-07-01

3.3 Problèmes rencontrés

Problème	Solution
Erreur 403 (Trustpilot bloquait)	Passage à Selenium
Chrome non trouvé (Brave par défaut)	Chemin Chrome indiqué manuellement
Clé JSON incorrecte	Script de debug → clé = businessUnits['businesses']
Boucle infinie à 1075 pages !	totalPages du JSON + détection des doublons
Catégories slug incorrects	Remplacement par slugs validés Trustpilot

✓ **RÉSULTAT : 10 664 entreprises réelles dans companies_trustpilot.csv**

4. Source 2 — Avis clients ShowroomPrivé

4.1 Stratégie de scraping (sans compte Trustpilot)

Trustpilot applique une limitation d'accès aux avis via la navigation sans authentification, avec un nombre de pages consultables restreint. Afin d'optimiser la collecte des données, une stratégie de filtrage par nombre d'étoiles a été mise en place.

Chaque filtre (de 1 à 5 étoiles) permet d'accéder à un ensemble d'avis indépendant, ce qui augmente le volume de données récupérables tout en respectant la structure d'accès disponible.

Filtre	Pages	Avis collectés
?stars=1 (avis 1 étoile)	10	134
?stars=2 (avis 2 étoiles)	10	51
?stars=3 (avis 3 étoiles)	10	55
?stars=4 (avis 4 étoiles)	10	172
?stars=5 (avis 5 étoiles)	10	180
TOTAL	50 pages	592 avis

4.2 Champs collectés par avis

Champ	Description
nb_etoiles	Note de 1 à 5
titre	Titre court du commentaire
contenu	Texte complet (verbatim)
entreprise_repond	L'entreprise a-t-elle répondu ?
reponse_entreprise	Texte de la réponse de l'entreprise

✓ **RÉSULTAT : 592 avis réels dans showroom_reviews.json**

Distribution : 134×1★ | 51×2★ | 55×3★ | 172×4★ | 180×5★

5. Publication sur GitHub

PARTIE BASE DE DONNEE

1. Contexte et objectif

La deuxième étape du pipeline de données est dédiée à l'intégration et à la structuration des données collectées. Les informations extraites sont nettoyées, normalisées puis chargées dans des systèmes de stockage afin de faciliter leur interrogation, leur exploitation analytique et leur utilisation future pour des modèles de Machine Learning.

Pourquoi deux bases différentes ?

PostgreSQL (SQL) → données structurées des entreprises (catégorie, trustscore, statistiques). Relations claires entre tables.

ElasticSearch (NoSQL) → commentaires clients en texte libre. Optimisé pour la recherche full-text et les analyses de sentiment.

Kibana → interface visuelle connectée à ElasticSearch pour visualiser les avis et suivre une entreprise.

2. Infrastructure Docker

2.1 Pourquoi Docker ?

Docker permet d'installer PostgreSQL, ElasticSearch et Kibana en une seule commande, sans configuration complexe. Tout tourne dans des conteneurs isolés sur le PC. C'est aussi ce que demande l'étape 4 du projet (dockeriser l'API).

2.2 Installation

- Téléchargement et installation de Docker Desktop sur Windows depuis docker.com
- Création d'un fichier docker-compose.yml qui définit les 3 services
- Lancement de tout en une seule commande :

```
docker-compose up -d
```

2.3 Les 3 conteneurs lancés

Conteneur	Image	Port	Rôle
postgres_satisfaction	postgres:15	5432	Base de données relationnelle
elasticsearch_satisfaction	elasticsearch:8.12.0	9200	Moteur de recherche NoSQL
kibana_satisfaction	kibana:8.12.0	5601	Dashboard visuel pour ES

2.4 Vérification

Après le lancement, vérification dans le navigateur :

- localhost:9200 → Elasticsearch répond avec version 8.12.0 ✓
- localhost:5601 → Kibana affiche 'Welcome home' ✓

✓ **Les 3 conteneurs Docker démarrés avec succès**
PostgreSQL + Elasticsearch + Kibana opérationnels

3. Base de données relationnelle (PostgreSQL)

3.1 Schéma — 3 tables

Nous avons séparé les données en 3 tables reliées entre elles par des clés étrangères :

Table	Colonnes principales	Lien
categorie	id, nom	—
entreprise	id, categorie_id, nom, slug	→ categorie
statistiques_avis	id, entreprise_id, trustscore, nb_avis_total, pct_excellent...pct_bad	→ entreprise

La séparation des données en plusieurs tables permet d'appliquer les principes de normalisation des bases relationnelles et d'éviter la duplication d'informations. Par exemple, une catégorie comme *"atm"* est enregistrée une seule fois dans la table dédiée aux catégories, puis référencée par les différentes entreprises associées. Ainsi, si 47 entreprises appartiennent à cette catégorie, l'information n'est pas répétée 47 fois, ce qui réduit le volume de données stockées et améliore la cohérence de la base.

3.2 Création des tables

Le script `create_database.sql` crée les tables dans PostgreSQL via Docker :

```
docker exec -i postgres_satisfaction psql -U postgres -d satisfaction_client < create_database.sql
```

Résultat : DROP TABLE (si existantes) + CREATE TABLE × 3 ✓

3.3 Chargement des données

Installation de la librairie Python :

```
pip install psycopg2-binary
```

Le script `load_data.py` assure le chargement des données collectées depuis le fichier `companies_trustpilot.csv` vers la base de données. Il lit chaque enregistrement du fichier source, effectue les transformations nécessaires puis répartit les informations dans les trois tables relationnelles dédiées afin de structurer les données et faciliter leur exploitation.

python load_data.py

- ✓ 3 catégories créées
- ✓ 10 664 entreprises insérées
- ✓ 10 664 lignes de statistiques insérées

3.4 Vérification avec des requêtes SQL

Le fichier queries.sql contient 6 requêtes de vérification. Exemple — trustscore moyen par catégorie :

```
SELECT categorie.nom, AVG(statistiques_avis.trustscore) AS trustscore_moyen
FROM categorie
JOIN entreprise ON entreprise.categorie_id = categorie.id
JOIN statistiques_avis ON statistiques_avis.entreprise_id = entreprise.id
GROUP BY categorie.nom;
```

Résultat obtenu :

Catégorie	Trustscore moyen	Interprétation
clothing_store	3.57	Les boutiques en ligne sont bien notées
courier_service	2.27	La livraison est problématique
atm	2.14	Les banques sont très mal notées

4. Base NoSQL (ElasticSearch)

4.1 Pourquoi ElasticSearch pour les avis ?

Les commentaires clients sont du texte libre. ElasticSearch est conçu pour stocker et rechercher dans du texte. Il s'intègre nativement avec Kibana pour créer des dashboards.

4.2 Création de l'index

Le fichier create_index_reviews.json définit les types de chaque champ (mapping) :

Champ	Type ES	Raison
review_id	keyword	Identifiant exact (filtre, tri)
company_name	keyword	Nom exact (agrégation par entreprise)
nb_etoiles	integer	Nombre (graphique distribution)
contenu	text	Texte analysé (recherche full-text, ML)
entreprise_repond	boolean	Booléen (taux de réponse)
date_avis	date	Date (évolution temporelle)

4.3 Problèmes rencontrés et solutions

Problème	Solution
elasticsearch-py v9 incompatible avec ES 8.12	Installation de la version compatible : pip install 'elasticsearch>=8,<9'
Syntaxe body= supprimée en v8	Remplacement par mappings=mapping['mappings']
Format date '2026-07-04 21:38:31' rejeté	Remplacement de l'espace par T avant insertion

4.4 Chargement des avis

```
python load_reviews_to_es.py
```

- ✓ 592 avis insérés dans ElasticSearch
- ✓ Index 'reviews' créé avec le bon mapping
- ✓ Vérification : localhost:9200/reviews/_count → 592

5. Dashboard Kibana

Kibana est accessible sur localhost:5601. Il permet de visualiser les données de l'index ElasticSearch avec des graphiques interactifs.

Visualisations préparées dans kibana_dashboard.html :

Visualisation	Ce qu'elle montre
Graphique barres — distribution étoiles	134×1★ 51×2★ 55×3★ 172×4★ 180×5★
Donut — taux de réponse	~18% des avis reçoivent une réponse de ShowroomPrivé
Tableau — avis négatifs sans réponse	Liste des clients insatisfaits à traiter en priorité

6. Bilan — Livrables produits

Fichier	Rôle
docker/docker-compose.yml	Lance PostgreSQL + ElasticSearch + Kibana en une commande
create_database.sql	Crée les 3 tables PostgreSQL
load_data.py	Charge companies_trustpilot.csv dans PostgreSQL
queries.sql	6 requêtes SQL de vérification
create_index_reviews.json	Mapping de l'index ElasticSearch
load_reviews_to_es.py	Charge showroom_reviews.json dans ElasticSearch
requetes_elasticsearch.txt	5 requêtes ES (recherche, filtre, agrégat)
kibana_dashboard.html	Dashboard visuel avec vrais chiffres (592 avis)
Explication_Architecture_Etape2.docx	Document d'architecture et de choix techniques

- ✓ ÉTAPE 2 TERMINÉE
- PostgreSQL : 10 664 entreprises + 3 catégories
- ElasticSearch : 592 avis clients indexés
- Kibana : dashboard opérationnel sur localhost:5601
- GitHub : tous les fichiers publiés et à jour

7. Prochaines actions

1. Passer le dashboard Kibana sur StreamLit (deadline : 15/08)
2. Automatiser le scrapping pour avoir encore plus de donnée live (deadline : 30/07)
3. Accroître le nombre de client (deadline : 30/08)
4. Cleaner le git (créer les dossiers propres => 15/07)
5. Mettre en place le ML sur les sentiments (deadline : 30/08)